

Informe Técnico Analítico

Trabajo Práctico Integrador 2026 — Sistema de Semáforos Inteligentes y Análisis Comparativo de Paradigmas

Grupo: 18 **Tema investigado (Fase 3):** Erlang **Repositorio:**

<https://github.com/valrom12/TPI-Funcional-2026-Grupo18>

Integrantes:

Nombre	Usuario de GitHub
Gonzalez, Rocío Anabel	@Anabel0710
Romani, Valentino	@valrom12
Gomez, Matías Gabriel	@MatiasGabriel
Lafuente, Noelia Magali	@noelialafuente18
Bereciartua Serpa, Osvaldo Agustín	@agustín-bereciartua

1. Introducción y objetivos

Este informe documenta el desarrollo del Trabajo Práctico Integrador correspondiente a la materia de Programación Funcional, cuyo objetivo es modelar el sistema de control de un semáforo urbano inteligente bajo el paradigma funcional puro. El trabajo se dividió en tres fases: (1) la implementación del núcleo lógico en Common Lisp respetando inmutabilidad, ausencia de bucles imperativos y clasificación taxonómica de funciones; (2) la integración de una librería externa del ecosistema Quicklisp para enriquecer el sistema de auditoría; y (3) la reimplementación comparativa de las funciones críticas (`transicion`) y (`timer`) en un segundo lenguaje funcional asignado de forma aleatoria —en nuestro caso, **Erlang**— con el fin de contrastar dos formas distintas de resolver el mismo problema lógico.

2. Fase 1 — Diseño funcional en Common Lisp

2.1 Arquitectura general

El núcleo (`lisp/core.lisp`) resuelve los siete requerimientos del enunciado base mediante funciones independientes que se componen entre sí, sin recurrir en ningún punto a estado mutable compartido. El color del semáforo nunca se "guarda": en cada llamada se deriva matemáticamente a partir del tiempo Unix recibido como argumento (`timer-semaforo`), y cada transición se calcula comparando el estado actual contra el estado destino (`transicion`), devolviendo siempre un nuevo valor sin alterar nada externo a la función.

22 Clasificación de funciones

La siguiente tabla resume la taxonomía declarada en los encabezados de comentario de cada función, tal como exige la consigna:

Función	Naturaleza	Estrategia de Control	Impacto en Memoria
transicion	Pura	Condicional	No destructiva
timer-semaforo	Pura	Condicional	No destructiva
auditoria-cambio	Impura (imprime en consola)	Secuencial	No destructiva
duracion-ciclo	Pura	Composición	No destructiva
recomendacion-ciclo	Pura	Condicional	No destructiva
ciclos-por-tiempo	Pura	Composición	No destructiva
distribucion-temporal	Pura	Composición	No destructiva
transicion-intermitente	Pura	Función Predicado	No destructiva
timer-semaforo-intermitente	Pura	Función Predicado	No destructiva
duracion-ciclo-intermitente	Pura	Función Simple	No destructiva
informe	Impura (escribe a archivo)	Funciones de Orden Superior(<code>mapc</code>)	No destructiva

23 Cumplimiento de las restricciones de diseño

- **Inmutabilidad absoluta:** no se utiliza `defparameter`, `defvar`, `setq` ni `self` en ningún punto del código. El estado del tráfico y del tiempo fluye exclusivamente como argumento de entrada de cada función(`tiempo`, `color-actual`, `cambiar-a`, etc.), y las únicas ligaduras locales se resuelven con `let`, que no muta nada externo.
- **Cero bucles imperativos:** no aparece `loop`, `dolist`, `dotimes` ni `while`. Dado que el problema es esencialmente de *clasificación por rango de tiempo* y no de *acumulación iterativa*, la mayoría de las funciones resuelven su lógica con `cond`. La única necesidad real de iterar sobre una colección aparece en la función `informe` (extensión de persistencia), donde se optó por `mapc` —función de orden superior— precisamente

por ser la alternativa no destructiva y declarativa a un bucle imperativo, aplicando un efecto secundario controlado (escritura a archivo) sin generar una lista intermedia en memoria.

- **Clasificación obligatoria:** las once funciones del núcleo incluyen el encabezado de comentario con Naturaleza, Estrategia de Control e Impacto en Memoria, según el formato exigido.

24 Extensiones (Iteración 2)

Más allá de los Requerimientos 1 a 6, se incorporaron dos extensiones:

- **Intermitencia de seguridad** (`transicion-intermitente`, `timer`, `semaforo-intermitente`, `duracion-ciclo-intermitente`): se introduce el estado `en-amarillo-intermitente` como paso obligatorio de 3 segundos entre cada cambio de fase, ampliando el ciclo total de 216 a 225 segundos ($90 + 3 + 120 + 3 + 6 + 3$).
- **Persistencia de datos** (`informe`): genera un archivo de texto (`informe-ejecucion-semaforo.txt`) con el historial de transiciones recibido como lista de registros, utilizando `with-open-file` para el manejo seguro del recurso y `mapc` para volcar cada registro sin mutar estructuras existentes.

25 Aseguramiento de la calidad (Requerimiento 7)

En lugar de una función dedicada, cada requerimiento incluye en el propio código sus casos de prueba documentados como comentarios bajo las categorías **Camino Normal**, **Camino Alternativo** y **Generación de Errores** (por ejemplo, en `duracion-ciclo` se documenta explícitamente que pasar una cadena de texto en lugar de un número provoca un error aritmético; en `informe` se documenta que pasar un átomo en lugar de una lista impide que `mapc` pueda iterar).

3. Fase 2 – Autonomía y ecosistema: Quicklisp + `local-time`

3.1 Justificación de la librería elegida

De las dos opciones propuestas por la cátedra (`local-time` y `cl-json`), el grupo eligió `local-time` para resolver el Requerimiento 3 (Sistema de Auditoría). La motivación fue que el sistema original reportaba el instante de cada cambio de color en formato Unix Epoch (un número de difícil lectura humana, por ejemplo `1700000000`), lo cual resulta poco útil para un reporte de auditoría real que debe ser legible por un operador humano.

3.2 Integración

La librería se carga al inicio del archivo con `(ql:quickload :local-time)`. Dentro de `auditoria-cambio`, el timestamp recibido se convierte primero a un objeto `timestamp` de la librería y luego se formatea como cadena legible:

```
lisp
```

```

(let ((fecha
      (local-time: format-timestring
        nil
        (local-time: unix-to-timestamp tiempo)
        :format '(: year "-" :month "-" :day " "
                  :hour ":" :min ":" :sec))))
  (format t "Tiempo ~A: la luz ha cambiado de ~A a ~A~%" fecha color-anterior c

```

3.3 Comparación antes / después

Salida

Antes (formato original)	Tiempo 1700000000: la luz ha cambiado de EN-ROJO a EN-VERDE
Después (con <code>local-time</code>)	Tiempo 2023-11-14 22:13:20: la luz ha cambiado de EN-ROJO a EN-VERDE

La integración fue íntegramente no destructiva: `auditoria-cambio` sigue siendo impura solo por su efecto de impresión (no por mutación de estado), y el resto del núcleo permanece inalterado.

4 Bitácora de depuración

#	Bug / Sintoma	Causa probable	Solución aplicada
1	<code>transicion no</code> reconocía un estado que parecía idéntico al esperado	Comparación de símbolos cotizados con <code>eq</code> en vez de <code>equal</code> (o viceversa), o diferencia de mayúsculas/minúsculas en el símbolo (Lisp normaliza a mayúsculas internamente)	Se usó <code>equal</code> de forma consistente para comparar los símbolos de color
2	Error al cargar el archivo: <code>local-time</code> no encontrado / paquete sin cargar	Quicklisp no estaba instalado/configurado en el <code>~/.sbclrc</code> , o se intentó usar <code>local-time</code> antes de ejecutar <code>(ql:quickload :local-time)</code>	Se verificó la instalación de Quicklisp y se aseguró que el <code>quickload</code> se ejecute como primera línea del archivo
3	<code>END-OF-FILE</code> o error de lectura al evaluar el archivo	Paréntesis sin cerrar en una función con <code>cond</code> anidado	Se recontaron los paréntesis con la ayuda del editor (resaltado de pareja) y se reorganizó la indentación
4	<code>ciclos-por-tiempo</code> devolvía un resultado inesperado (decimal en vez de entero, o error de tipo)	<code>floor</code> recibiendo una división que no resolvía a un valor numérico esperado, o confusión entre <code>/</code> (división racional exacta) y división entera	Se ajustó el orden de las operaciones aritméticas y se verificó el tipo de retorno con <code>(type-of ...)</code> en el REPL

5. Fase 3 — Estudio comparativo: Erlang

5.1 Presentación del lenguaje

Erlang es un lenguaje funcional concurrente creado en 1986 por Joe Armstrong, Robert Virding y Mike Williams en los laboratorios de Ericsson, con el objetivo original de controlar centrales telefónicas que debían operar de forma ininterrumpida. Corre sobre su propia máquina virtual, la **BEAM**, y se caracteriza por el **modelo de actores** (procesos livianos y aislados que se comunican exclusivamente por paso de mensajes), la **tolerancia a fallos** mediante la filosofía "let it crash" (dejar que un proceso falle y sea reiniciado por un

supervisor en vez de intentar prevenir cada error), y variables de **asignación única** (cada variable se liga una sola vez dentro de su ámbito).

5.2 Industrias y empresas que lo utilizan

- **Telecomunicaciones:** Ericsson, su creador original, lo sigue usando en infraestructura de redes y centrales de conmutación.
- **Mensajería instantánea:** WhatsApp construyó su backend de mensajería sobre Erlang, lo que le permitió manejar millones de conexiones concurrentes con pocos servidores.
- **Fintech:** Klarna utiliza Erlang en partes críticas de su plataforma de pagos por su tolerancia a fallos.
- **Infraestructura de mensajería:** RabbitMQ (uno de los *brokers* de mensajería más usados en la industria) y ejabberd (servidor de mensajería XMPP) están escritos en Erlang.
- **Apuestas en tiempo real:** empresas del sector de apuestas deportivas en vivo (como Bet365) lo emplean por su capacidad de manejar miles de eventos concurrentes con baja latencia.

5.3 Traducción de las funciones transición y timer

El archivo comparativa/solucion.erl reimplementa únicamente las dos funciones críticas, manteniendo el mismo encabezado de clasificación exigido por la cátedra:

```
erlang
```

```
- module( solucion ) .  
- export([transicion/2, timer/1]) .
```

```
%%% FUNCION: transicion  
%%% NATURALEZA: Pura  
%%% ESTRATEGIA: Condicional  
%%% IMPACTO: No destructiva
```

```
transicion(en_rojo, verde) ->  
    [en_rojo, cambiar_a_verde];  
transicion(en_verde, amarillo) ->  
    [en_verde, cambiar_a_amarillo];  
transicion(en_amarillo, rojo) ->  
    [en_amarillo, cambiar_a_rojo];  
transicion(ColorActual, _) ->  
    [ColorActual, accion_por_defecto].
```

```
%%% FUNCION: timer  
%%% NATURALEZA: Pura  
%%% ESTRATEGIA: Condicional  
%%% IMPACTO: No destructiva
```

```
timer(TiempoUnix) ->  
    Resto = TiempoUnix rem 216, if  
        Resto < 90 -> en_rojo;  
        Resto < 210 -> en_verde; true -  
        > en_amarillo  
end.
```

`transicion/2` resuelve por **cláusulas múltiples con pattern matching** lo que en Lisp se resolvía con un único `cond`: en lugar de evaluar condiciones dentro del cuerpo de la función, Erlang selecciona la cláusula cuya cabecera coincide con los argumentos recibidos. `timer/1` mantiene la misma lógica matemática que su contraparte en Lisp (residuo del tiempo Unix módulo la duración total del ciclo), expresada con una construcción `if` de guardas.

5.4 Preguntas teóricas — Erlang

1. Sintaxis derivada de Prolog: delimitación de funciones y expresiones

En Erlang, cada función puede tener varias **cláusulas** (una por cada patrón de entrada distinto), y la puntuación indica exactamente dónde termina cada cosa: una **coma** (`,`) separa expresiones secuenciales dentro de una misma cláusula (por ejemplo, dentro de `timer/1`), la asignación `Resto = TiempoUnix rem 216,` termina en coma porque a continuación viene la expresión `if`); un **punto y coma** (`;`) separa cláusulas alternativas de

la misma función, o ramas alternativas dentro de un `if`/`case` (las primeras tres cláusulas de `transicion/2` terminan en `;`); y un **punto** (`.`) cierra por completo la definición de la función, indicando "no hay más cláusulas". En nuestro código, la última cláusula de `transicion/2` (`(transicion(ColorActual, _) -> [ColorActual, accion_por_defecto].)`) termina en punto precisamente porque es la última.

Esto afectó la lectura del código de una manera muy distinta a Lisp: en Common Lisp el delimitador es siempre el mismo símbolo (el paréntesis), y la dificultad es puramente de balanceo —cualquier editor con resaltado de pares lo señala de inmediato—. En Erlang, en cambio, el símbolo correcto a usar **depende del contexto sintáctico** (¿es la última cláusula? ¿es la última expresión del cuerpo?), por lo que un error de puntuación no abre un paréntesis sin cerrar, sino que cambia el significado del programa o impide que compile, y al principio fue una fuente frecuente de errores de compilación al traducir las funciones desde Lisp.

2 Variables de asignación única y simulación del ciclo continuo

En Erlang, una vez que una variable se liga a un valor dentro de su ámbito, no puede volver a ligarse a otro valor distinto (intentarlo provoca un error de *badmatch*). Esto impide el patrón típico de un lenguaje imperativo de "actualizar una variable de estado en cada iteración del bucle".

En nuestra solución, esto no representa un problema porque `timer/1` está diseñada como una **función pura sin estado interno**: en cada invocación recibe el tiempo Unix actual como argumento y *recalcula* el color correspondiente desde cero (mediante el resto de la división), en vez de mantener un contador que se vaya incrementando y reasignando. La "continuidad" del semáforo no se logra mutando una variable a través del tiempo, sino llamando repetidamente a una función pura con un nuevo valor de tiempo cada vez —la responsabilidad de "avanzar el reloj" queda fuera de la función, en quien la invoca—.

El patrón general en Erlang para un ciclo continuo con cambios de estado (por ejemplo, si quisiéramos modelar el semáforo como un proceso activo que vive en el tiempo) es la **recursión de cola sobre un proceso**: en vez de reasignar una variable, el proceso se llama a sí mismo pasando el *nuevo* estado como argumento de la siguiente invocación (`loop(NuevoEstado)`), de modo que cada "vuelta del ciclo" introduce una ligadura completamente nueva en un nuevo marco de ejecución, en lugar de mutar la existente.

5.5 Conclusión del grupo sobre la experiencia con Erlang

Trabajar con Erlang después de Common Lisp permitió notar que ambos lenguajes comparten la base funcional (funciones puras, inmutabilidad, recursión), pero la difieren radicalmente en cómo se expresa el control de flujo: donde Lisp resuelve todo dentro del cuerpo de una única función con `cond`, Erlang lo reparte en múltiples cláusulas que se

seleccionan por coincidencia de patrón, lo cual resultó, una vez superada la curva de aprendizaje de la puntuación, más declarativo y cercano a una tabla de verdad. La principal dificultad inicial fue puramente sintáctica (la combinación de comas, puntos y puntos y coma), mientras que el modelo de datos y la prohibición de mutabilidad no representaron un cambio conceptual grande respecto de las restricciones ya impuestas en la Fase 1 de Lisp.

6. Conclusiones generales del trabajo práctico

El desarrollo de este TPI permitió aplicar de forma concreta los pilares del paradigma funcional —pureza, inmutabilidad y composición— sobre un problema con sentido real (el control de un semáforo urbano), evidenciando que incluso un sistema con comportamiento dependiente del tiempo puede modelarse sin un solo bit de estado mutable, derivando siempre el resultado a partir de los argumentos recibidos. La incorporación de `local-time` mostró además cómo el ecosistema de Quicklisp permite extender Common Lisp con funcionalidad de calidad productiva sin romper ninguna de las restricciones de diseño funcional. Finalmente, la comparación con Erlang reforzó la idea de que el paradigma funcional no es una sintaxis única sino una familia de soluciones —pattern matching por cláusulas frente a `cond`, procesos con paso de mensajes frente a llamadas directas— que comparten los mismos principios de fondo aun cuando se expresan de maneras muy distintas.

7. Bibliografía

- Enunciado base: *Ejercicio Semáforos v2.1* — Documento de la cátedra.
- Common Lisp HyperSpec.
[http: /www.lispworks.com/documentation/HyperSpec/Front/](http://www.lispworks.com/documentation/HyperSpec/Front/)
- Quicklisp. [https: /www.quicklisp.org/](https://www.quicklisp.org/)
- `local-time` (librería de Quicklisp). [https: /github.com/dlowe-net/local-time](https://github.com/dlowe-net/local-time)
- Erlang — Sitio oficial. [https: /www.erlang.org/](https://www.erlang.org/)
- Try Erlang. [https: /www.tryerlang.org/](https://www.tryerlang.org/)
- *[Agregar aquí cualquier otra fuente consultada por el grupo]*